Session 27 — Live Code Walkthrough

This is a friendly, step-by-step guide to everything we built together in class during Session 27. It covers every LIVE step, from LC-01 to LC-19, in the order we actually built them.

After reading this, you should be able to rebuild the whole Comments + `@mention` feature yourself, and explain *why* each piece works, not just *what* it does.

By the end of LIVE, our app can do three new things:

1. Post and read **comments** on an issue.
2. Render a stored `@mention` as a nice little blue pill, like `@Omar Rahal`.
3. Type `@` inside a comment and get a real autocomplete dropdown of your teammates, with the cursor landing in exactly the right place after you pick someone.

Part A — Comments (LC-01 to LC-08)

Step LC-01 — A `Comment` type

Goal: Give TypeScript a shape for what a comment looks like, before we write any component that uses one.

File: `web/src/lib/types.ts`

What we add:

TSX

```tsx
export type Comment = {
  id: number;
  issueId: number;
  authorId: number | null;
  body: string;
  createdAt: string;
  author: { id: number; name: string } | null;
};
```

How it works: This is just a description of the shape of the data, written in TypeScript. It doesn't run any code — it just tells the editor and the compiler "whenever you see a `Comment`, expect it to have these fields."

Why we need it: We already do this for every other piece of data in the app (`Issue`, `Sprint`, and so on). Adding the type first, before writing the components that use it, means TypeScript can catch our mistakes early — for example, if we try to read `comment.autorId` (misspelled), TypeScript will complain immediately.

One detail worth slowing down on: `author` can be `null`. Why? Because if the person who wrote a comment later has their account deleted, the comment itself should NOT disappear or crash the page — it should just show "Deleted user" instead of a name. That's why `author` is `{ id, name } | null`, not just `{ id, name }`.

What to test: `npx tsc --noEmit` — this should pass with no errors. Nothing uses `Comment` yet, so there's nothing to see in the browser.

Step LC-02 — `createCommentAction`, the validation part

Goal: Start writing the Server Action that will create a comment — but only the part that checks the form is filled in correctly. We'll add the real network call in the next step.

File: `web/src/lib/actions/comments.ts` (new file)

What we add:

TSX

```tsx
"use server";

import { requireSessionToken } from "@/lib/session";

export interface CommentFormState {
  error?: string;
}

export async function createCommentAction(
  issueId: number,
  basePath: string,
  _prevState: CommentFormState,
  formData: FormData,
): Promise<CommentFormState> {
  const token = await requireSessionToken();
  const body = String(formData.get("body") ?? "").trim();

  if (!body) {
    return { error: "Comment cannot be empty." };
  }

  // The real backend call comes next (LC-03).
  return {};
}
```

How it works: `"use server"` at the top of the file tells Next.js that every function exported from this file runs on the server, never in the browser. A **Server Action** is just a normal async function that the browser is allowed to call directly, almost like calling an API endpoint — except you call it like a regular function, and Next.js handles the network request for you behind the scenes.

Look at the four parameters: `issueId`, `basePath`, `_prevState`, `formData`. The last two (`_prevState` and `formData`) are supplied automatically by React's `useActionState` hook, which we'll wire up in LC-04. The first two (`issueId`, `basePath`) are extra information *we* need to pass in ourselves — we'll do that using `.bind()`, which you'll see in LC-04.

Why we need it: This is the exact same shape we already used for creating issues and sprints. We're not inventing a new pattern — we're reusing one you already know. Recognizing "this is the same shape as before" is a real skill, and it's often more useful than memorizing new syntax.

What to test: `npx tsc --noEmit`. Clean exit expected. Nothing calls this function yet.

Step LC-03 — `createCommentAction`, the real backend call

Goal: Actually send the comment to the backend and create it for real.

File: `web/src/lib/actions/comments.ts`

What we add:

TSX

```tsx
import { revalidatePath } from "next/cache";
import { apiFetch } from "@/lib/api";
import { requireSessionToken } from "@/lib/session";
```

TSX

```tsx
  const res = await apiFetch(`/issues/${issueId}/comments`, {
    method: "POST",
    token,
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ body }),
  });

  if (!res.ok) {
    return { error: "Something went wrong. Please try again." };
  }

  revalidatePath(basePath);
  return {};
}
```

How it works: `apiFetch` sends a `POST` request to our NestJS backend, asking it to create a comment on this issue. If the backend says "no" (any non-2xx response), we return an error message that the form can show the user. If it says "yes," we call `revalidatePath(basePath)`.

What does `revalidatePath` actually do? Think of it in four steps:

TEXT

```
backend data changed
↓
revalidatePath tells Next.js "the data behind this page is now stale"
↓
the next time this page is requested, Next.js fetches fresh data instead of using a cached
copy
```

```
↓
the user sees the new comment, without us writing any extra state-management code
```

Why we need it: We never read the actual response body here (the new comment object). Why not? Because `revalidatePath` is going to re-fetch the whole page anyway, including the new comment — so reading the response ourselves would be wasted work. Any `@mention` notifications triggered by this comment already happened on the backend, inside the same request — the frontend doesn't need to know or do anything about that.

What to test:

BASH

```
curl -s -X POST http://localhost:3000/issues/1/comments -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" -d '{"body":"A real test comment."}'
```

Expected result: The comment text comes back in the response — proof the backend really created it. Nothing in the UI calls this function yet.

Step LC-04 — `CommentForm`, the skeleton

Goal: Create the form component and wire it to `createCommentAction` with `useActionState` — no visible fields yet.

File:
`web/src/app/(app)/workspaces/[id]/projects/[projectId]/issues/[issueId]/CommentForm.tsx` (new file)

What we add:

TSX

```
"use client";

import { useActionState } from "react";
import { createCommentAction, type CommentFormState } from "@/lib/actions/comments";

const initialState: CommentFormState = {};

export function CommentForm({
  issueId,
  basePath,
}: {
  issueId: number;
  basePath: string;
}) {
  const action = createCommentAction.bind(null, issueId, basePath);
  const [state, formAction, pending] = useActionState(action, initialState);
```

```
  return <form action={formAction} className="flex flex-col gap-2" />;
}
```

How it works: This is where `.bind()` comes in, answering the question from LC-02. `createCommentAction.bind(null, issueId, basePath)` creates a new version of the function that already has `issueId` and `basePath` "baked in" — so when React calls it later with `(prevState, formData)`, the full real call ends up being `createCommentAction(issueId, basePath, prevState, formData)`, exactly matching the signature we wrote in LC-02.

`useActionState(action, initialState)` gives us three things:

- `state` — whatever the action last returned (so far, `{}`, meaning no error).
- `formAction` — a special version of `action` you plug directly into `<form action={...}>`.
- `pending` — `true` while the action is running, so we can disable the submit button.

Why we need it: This is identical to how `CreateIssueForm` and `CreateSprintForm` already work. Same shape, new resource.

What to test: `npx tsc --noEmit`. Clean exit. Nothing renders this component yet.

Step LC-05 — `CommentForm`, the real fields

Goal: Add the actual textarea, an error message, and a submit button.

File: Same `CommentForm.tsx`

What we add:

TSX

```
  return (
    <form action={formAction} className="flex flex-col gap-2">
      {state.error && (
        <p className="text-sm text-red-600 dark:text-red-400">{state.error}</p>
      )}
      <textarea
        name="body"
        rows={2}
        required
        placeholder="Add a comment..."
        className="rounded-md border border-gray-300 px-3 py-1 text-sm dark:border-gray-600
dark:bg-gray-800"
      />
      <button
        type="submit"
        disabled={pending}
        className="rounded-md border border-blue-600 bg-blue-600 px-3 py-1 text-sm font-
medium text-white disabled:opacity-50"
      >
        {pending ? "Posting..." : "Comment"}
```

```
      </button>
    </form>
  );
```

How it works: The `<textarea name="body">` is a plain, "uncontrolled" field — React doesn't track its value in state; the browser just manages it, and `formData.get("body")` reads it when the form submits. That's intentionally simple for now, because later (LC-12 onward) we're going to replace this exact field with a smarter, controlled one that can detect `@mentions`. Building the plain version first, and proving it works, keeps this step small.

Why we need it: `{state.error && ...}` only shows the error paragraph when there actually is one. `disabled={pending}` stops a user from double-submitting while the request is in flight.

What to test: `npx tsc --noEmit`. Clean exit. Still nothing renders this component.

Step LC-06 — Wire `CommentForm` into Issue Detail

Goal: Actually show the comment form on the Issue Detail page.

File: `.../issues/[issueId]/page.tsx`

What we add:

TSX

```
import { CommentForm } from "./CommentForm";
```

TSX

```
      </Card>
      <CommentForm issueId={issue.id} basePath=
{`/workspaces/${id}/projects/${projectId}/issues/${issueId}`} />
    </main>
  );
}
```

How it works: `basePath` is the exact URL of the page we're on. Why do we pass it in, instead of hard-coding it inside the action? Because `revalidatePath` needs to know exactly *which* page to refresh — and the same `CommentForm` component could in theory be used on a different page with a different URL. Passing `basePath` in as a prop keeps the component reusable.

Why we need it: Without this step, the form exists but nobody can see it.

What to test:

BASH

```
curl -s -X POST http://localhost:3000/issues/1/comments -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" -d '{"body":"A real test comment."}'
```

Expected result: A signed-in user visiting Issue Detail can now actually see and submit the comment form.

Step LC-07 — `CommentList`, the skeleton

Goal: Build a component that lists every comment on an issue, plain text for now.

File: `.../issues/[issueId]/CommentList.tsx` (new file)

What we add:

TSX

```tsx
import type { Comment } from "@/lib/types";

export function CommentList({ comments }: { comments: Comment[] }) {
  if (comments.length === 0) {
    return <p className="text-sm text-gray-400">No comments yet.</p>;
  }

  return (
    <ul className="flex flex-col gap-2">
      {comments.map((c) => (
        <li key={c.id} className="rounded-md bg-gray-50 p-2 text-sm dark:bg-gray-900">
          <p className="font-medium">{c.author?.name ?? "Deleted user"}</p>
          <p className="whitespace-pre-wrap">{c.body}</p>
        </li>
      ))}
    </ul>
  );
}
```

How it works: `comments.map(...)` turns our array of comments into an array of `<li>` elements. `c.author?.name ?? "Deleted user"` is the payoff from LC-01's nullable `author` field — if `author` is `null` (because that user's account was removed), we fall back to a safe string instead of crashing.

Why we need it: Reading `c.author.name` directly (without the `?.`) would throw a runtime error the one time a comment's author account was actually deleted. The `?.` protects us.

What to test: `npx tsc --noEmit`. Clean exit. Nothing renders this component yet, and nothing fetches real comments yet.

Step LC-08 — Wire `CommentList` into Issue Detail

Goal: Fetch the real comments and actually show them on the page.

File: `.../issues/[issueId]/page.tsx`

What we add:

TSX

```tsx
import type { Issue, Project, Comment } from "@/lib/types";
```

TSX

```tsx
import { CommentForm } from "./CommentForm";
import { CommentList } from "./CommentList";
```

TSX

```tsx
  const membersRes = await apiFetch(`/workspaces/${project.workspaceId}/members`, { token
});
  const members: Member[] = await membersRes.json();

  const commentsRes = await apiFetch(`/issues/${issue.id}/comments`, { token });
  const comments: Comment[] = await commentsRes.json();
```

TSX

```tsx
      <CommentList comments={comments} />
      <CommentForm issueId={issue.id} basePath=
{`/workspaces/${id}/projects/${projectId}/issues/${issueId}`} />
    </main>
  );
}
```

How it works: This is a Server Component (the whole page), so the `await apiFetch(...)` call happens on the server, before the page is ever sent to the browser. We pass `{ token }` because the backend route requires the visitor to be signed in.

Why we need it: Without `{ token }`, the backend would reject the request with `401 Unauthorized` — the comments route is protected, just like every other real data route in this app.

What to test: After posting a comment with the form, reload the page and confirm it's still there (this is what proves it's really saved on the backend, not just shown locally).

What you should see after this step: Comments can now be created and read, end to end. There's just no way yet to reference a teammate inside one, or to make the mention look nice. That's what the rest of this document covers.

Part B — Rendering a stored `@mention` (LC-09, LC-10)

Before we build the autocomplete that *creates* a mention, let's look at how a mention is *stored* and *displayed*, since that's simpler and it'll make the autocomplete easier to understand later.

A comment's text is stored as plain text, with a small, deliberately simple bit of markup embedded in it — never a rich-text document. When someone mentions a teammate, the stored text looks like this:

TEXT

```
@[Nawar](12)
```

But what the reader actually sees on screen is:

TEXT

```
@Nawar
```

styled as a little blue pill. `MentionText` is the one component responsible for turning the first thing into the second.

Step LC-09 — `MentionText`

Goal: Build a component that finds every `@[Name](id)` token in a comment and turns it into a styled pill — not used anywhere yet.

File: `web/src/components/ui/MentionText.tsx` (new file)

What we add:

TSX

```tsx
const MENTION_RE = /@\[([^\]]+)\]\(\d+\)/g;

export function MentionText({ text }: { text: string }) {
  const parts: React.ReactNode[] = [];
  let lastIndex = 0;
  let match: RegExpExecArray | null;
  let key = 0;

  MENTION_RE.lastIndex = 0;
  while ((match = MENTION_RE.exec(text)) !== null) {
    if (match.index > lastIndex) {
      parts.push(text.slice(lastIndex, match.index));
    }
    parts.push(
      <span key={key++} className="rounded bg-blue-50 px-1 font-medium text-blue-700
dark:bg-blue-950 dark:text-blue-300">
        @{match[1]}
      </span>,
    );
    lastIndex = match.index + match[0].length;
  }
  if (lastIndex < text.length) {
    parts.push(text.slice(lastIndex));
  }

  return <>{parts}</>;
}
```

How it works, piece by piece. This is one of the two "hard" parts of LIVE, so let's take it slowly. We'll use this example sentence:

TEXT

```
Hello @[Nawar](12), please ask @[Ali](7)
```

The job of this component is to turn that one string into an array of pieces React can render — some pieces are plain text, some are styled `<span>` pills:

TEXT

```
[
  "Hello ",
  <span>@Nawar</span>,
  ", please ask ",
  <span>@Ali</span>
]
```

Here's what each variable means:

Name	What it holds
`MENTION_RE`	The pattern that finds a `@[Name](id)` token anywhere in the text.
`parts`	The array we're building up — a mix of plain strings and `<span>` elements.
`lastIndex`	How far into the text we've already copied into `parts`.
`match`	The result of the most recent regex match, or `null` when there are no more.
`match.index`	Where in the string this match *starts*.
`match[0]`	The whole matched text, e.g. `"@[Nawar](12)"`.
`match[1]`	Just the part inside the first `(...)` group in the regex — the name, e.g. `"Nawar"`.

Now walk through the loop with our example string:

1. `MENTION_RE.exec(text)` finds `@[Nawar](12)` starting at index 6. Since `match.index` (6) is greater than `lastIndex` (0), we first push the plain text *before* the match: `"Hello "`.
2. We push a styled `<span>@Nawar</span>` pill — built from `match[1]`, which is just `"Nawar"`. Notice we never use `match[0]` here, and we never use the id number inside the parentheses — the id's only job is to be a stable reference; nothing about *displaying* the mention needs it.
3. We update `lastIndex` to point right after the match, so the next piece of plain text starts there.
4. The loop runs again, finds `@[Ali](7)`, and repeats: push the plain text in between (`", please ask "`), then push the `@Ali` pill.
5. When the loop finds no more matches, we push whatever plain text is left after the very last match — in a real comment, this is often empty, but it won't always be.

The result is exactly the five-item array shown above, and React is perfectly happy rendering an array of mixed strings and elements like that.

Why `MENTION_RE.lastIndex = 0`? `MENTION_RE` is declared once, outside the component, so it gets reused across every comment rendered. A regex with the global (`g`) flag remembers where it left off between calls — so without resetting `lastIndex` back to `0` first, the *second* comment we ever render

could start scanning from wherever the *first* comment's search happened to stop, silently missing mentions near the start of the text.

Why we need it: Without this component, every comment would show the raw `@[Name](id)` text to the reader, which is ugly and confusing.

What to test: `npx tsc --noEmit`. Clean exit. Nothing renders this component yet.

Step LC-10 — Wire `MentionText` into `CommentList`

Goal: Make every comment actually use `MentionText` to render its body.

File: `CommentList.tsx`

What we add:

TSX

```
import { MentionText } from "@/components/ui/MentionText";
```

TSX

```
          <p className="whitespace-pre-wrap">
            <MentionText text={c.body} />
          </p>
```

How it works: We just swap out `{c.body}` (plain text) for `<MentionText text={c.body} />` (parsed, styled text). The surrounding `<p>` tag and its styling don't change at all.

What to test:

BASH

```
curl -s -X POST http://localhost:3000/issues/1/comments -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" -d '{"body":"cc @[Omar Rahal](2)"}'
```

Expected result: The comment renders with `@Omar Rahal` shown as a styled pill, not the raw `@[Omar Rahal](2)` text.

Part C — The `@mention` autocomplete (LC-11 to LC-18)

This is the biggest, newest, and hardest part of Session 27. Take your time with this section.

The feature we're building: while typing a comment, if you type `@` followed by some letters, a dropdown appears showing matching teammates. Click one, and it gets inserted as a `@[Name](id)` token, right where you were typing — with your cursor landing in the correct spot afterward so you can keep typing normally.

Step LC-11 — `searchUsersAction` learns a new trick

Goal: Teach the existing user-search action to search for people who ARE already members (not just people who could be invited).

File: `web/src/lib/actions/workspace-members.ts`

What we add:

TSX

```
export async function searchUsersAction(
  workspaceId: number,
  q: string,
  membersOnly = false,
) {
  if (q.trim().length < 2) return [];
  const token = await requireSessionToken();
  const membersOnlyQuery = membersOnly ? "&membersOnly=true" : "";
  const res = await apiFetch(
    `/users/search?
q=${encodeURIComponent(q)}&workspaceId=${workspaceId}${membersOnlyQuery}`,
    { token },
  );
```

How it works: We already had this action — it's used by the "Add Member" picker, which searches for people who are NOT already a member (so they can be invited). A mention picker needs the exact opposite: only an existing member can genuinely be mentioned, since you can only mention someone who's actually in the workspace. The backend's own search route already supports both directions through one query parameter, `membersOnly` — we just needed to let the frontend action pass it through.

Why `membersOnly = false` as a default, instead of a required third argument? Because the "Add Member" picker already calls `searchUsersAction(workspaceId, q)` with only two arguments, and we don't want to break that existing code. Giving the new parameter a default value means every old call site keeps working exactly as before, and only the new mention picker needs to pass `true`.

What to test: `npx tsc --noEmit`. Clean exit. Nothing passes `true` yet — that comes in LC-14.

Step LC-12 — `MentionTextarea`, the skeleton

Goal: Start a brand-new component: a controlled textarea, with no mention detection yet.

File: `web/src/components/ui/MentionTextarea.tsx` (new file)

What we add:

TSX

```
"use client";

import { useState } from "react";

export function MentionTextarea({
  name,
  workspaceId,
  required = false,
}: {
  name: string;
  workspaceId: number;
```

```
  required?: boolean;
}) {
  const [value, setValue] = useState("");

  return (
    <textarea
      name={name}
      value={value}
      onChange={(e) => setValue(e.target.value)}
      rows={2}
      required={required}
      placeholder="Add a comment... (type @ to mention someone)"
      className="rounded-md border border-gray-300 px-3 py-1 text-sm dark:border-gray-600
dark:bg-gray-800"
    />
  );
}
```

How it works: Remember LC-05's plain textarea, where React didn't track the value at all? This one is different — it's a **controlled** textarea. `value={value}` means React is now the single source of truth for what's in the box, and `onChange` keeps `value` in sync with every keystroke.

Why does this matter? Because the mention picker needs to track the text AND the cursor position together, so that later it can insert a token at the *exact* spot the user was typing. A plain, uncontrolled textarea can't give us that level of control — a controlled one can.

Why accept `workspaceId` now, even though nothing uses it yet? The search we're about to add (LC-14) will need it. Declaring the full shape of the props upfront, even before every prop is used, is a convention we've used before in this app.

What to test: `npx tsc --noEmit`. Clean exit. Typing `@` does nothing special yet.

Step LC-13 — Detecting an in-progress `@query`

Goal: After every keystroke, figure out whether the user is in the middle of typing a mention right at the cursor.

File: `MentionTextarea.tsx`

What we add:

TSX

```
// Matches an @-mention being typed right at the cursor, e.g. "...cc @dan" -- captures the
// partial name so far. Requires a word boundary or start of text before the @, so an
// email-like "foo@bar" never triggers it.
const TRAILING_MENTION_RE = /(?:^|\s)@(\w{0,30})$/;
```

TSX

```
  const [value, setValue] = useState("");
  const [query, setQuery] = useState<string | null>(null);
```

```
  const [mentionStart, setMentionStart] = useState<number | null>(null);

  function handleChange(e: React.ChangeEvent<HTMLTextAreaElement>) {
    const text = e.target.value;
    const cursor = e.target.selectionStart ?? text.length;
    setValue(text);

    const match = TRAILING_MENTION_RE.exec(text.slice(0, cursor));
    if (match) {
      setMentionStart(cursor - match[1].length - 1);
      setQuery(match[1]);
    } else {
      setMentionStart(null);
      setQuery(null);
    }
  }
```

TSX

```
      onChange={handleChange}
```

How it works. This is the other "hard" part of LIVE, so let's break it down fully.

The problem we're solving: `"...cc @dan"` IS someone starting a mention. `"foo@bar"` is an email-like string and must NEVER trigger the dropdown. We need a rule that tells these two apart.

The regex, in plain English:

TEXT

```
/(?:^|\s)@(\w{0,30})$/
```

- `(?:^|\s)` — right before the `@`, there must be either the very start of the text, OR a whitespace character. This is exactly what rules out `"foo@bar"` — there's a letter (`o`) right before that `@`, not whitespace or the start of the string.
- `@` — the literal `@` character itself.
- `(\w{0,30})` — then, zero to 30 "word" characters (letters, digits, underscore) — this is the partial name the user has typed so far, like `"da"` while typing `"dan"`. This part is wrapped in parentheses, which means the regex engine remembers it separately — that's what `match[1]` will hold.
- `$` — and this whole pattern must end exactly at the position we're checking.

Why `text.slice(0, cursor)` instead of searching the whole textarea? Picture a comment where the cursor is in the *middle* of existing text:

TEXT

```
Thanks @dan for the help, I'll ask @nawar tomorrow
             ^
         cursor is here, right after "dan"
```

If we searched the *whole* `text`, the regex (with the `$` anchor) would only ever match something ending at the very end of the whole string — it would never notice the `@dan` the user is actually editing in the middle. By slicing the text down to `text.slice(0, cursor)` first — just `"Thanks @dan"` — the `$` anchor now means "ends exactly at the cursor," which is exactly the question we want answered: *is the user, right now, in the middle of typing a mention?*

How `mentionStart` is calculated:

TSX

```
mentionStart = cursor - match[1].length - 1
```

Walk through it with `"Thanks @dan"` and a cursor at position 11 (right after the "n" in "dan"):

- `match[1]` is `"dan"`, so `match[1].length` is `3`.
- `cursor - match[1].length` = `11 - 3` = `8` → that's the position of `d`, not the `@`.
- Subtract one more for the `@` itself: `8 - 1` = `7` → that's exactly where the `@` sits.

So `mentionStart` always points at the `@` character itself, not at the first letter of the name. We need that exact position later, in LC-16, to know where to start replacing text.

Why we need it: Without this detection step, we have no way of knowing when to search for users or show a dropdown.

What to test: `npx tsc --noEmit`. Clean exit. `query` / `mentionStart` aren't read from anywhere yet.

Step LC-14 — Actually searching for members

Goal: Whenever `query` is set, search for matching workspace members.

File: `MentionTextarea.tsx`

What we add:

TSX

```
import { useEffect, useState } from "react";
import { searchUsersAction } from "@/lib/actions/workspace-members";
```

TSX

```
  const [mentionStart, setMentionStart] = useState<number | null>(null);
  const [results, setResults] = useState<{ id: number; name: string }[]>([]);

  useEffect(() => {
    if (query === null) {
      setResults([]);
      return;
    }
    searchUsersAction(workspaceId, query, true).then(setResults);
  }, [query, workspaceId]);
```

How it works: This `useEffect` re-runs any time `query` or `workspaceId` changes — that's what the dependency array `[query, workspaceId]` means. Every time the user types another character while in the middle of a mention, `query` changes, and this effect fires again, searching fresh.

Why the third argument is `true`: Remember LC-11's `membersOnly` parameter? Passing `true` here means: only search people who are already real members of this workspace, since those are the only valid mention targets.

What to test: `npx tsc --noEmit`. Clean exit. `results` isn't rendered anywhere yet.

Step LC-15 — The results dropdown

Goal: Actually show the matching members in a dropdown list.

File: `MentionTextarea.tsx`

What we add:

TSX

```
  return (
    <div className="relative">
      <textarea
        name={name}
        value={value}
        onChange={handleChange}
        rows={2}
        required={required}
        placeholder="Add a comment... (type @ to mention someone)"
        className="rounded-md border border-gray-300 px-3 py-1 text-sm dark:border-gray-600
dark:bg-gray-800"
      />
      {query !== null && results.length > 0 && (
        <ul className="absolute z-10 mt-1 w-64 rounded-md border border-gray-200 bg-white p-
1 shadow-lg dark:border-gray-700 dark:bg-gray-800">
          {results.map((user) => (
            <li key={user.id}>
              <button type="button" className="w-full rounded px-2 py-1 text-left text-sm
hover:bg-gray-100 dark:hover:bg-gray-700">
                {user.name}
              </button>
            </li>
          ))}
        </ul>
      )}
    </div>
  );
```

How it works: `query !== null && results.length > 0` means the dropdown only shows when the user is actively mid-mention AND there's at least one matching person. The outer `<div className="relative">` exists purely so the dropdown (`absolute`) can position itself relative to the textarea, instead of relative to the whole page.

Why each result is a `<button type="button">`, not a submit button: If we used the default button type inside a `<form>`, clicking a result would accidentally submit the whole comment form. `type="button"` tells the browser "this button does NOT submit any form," so clicking a teammate's name only runs our own `onClick` handler (added next, in LC-16) — it never accidentally posts the comment.

What to test: `npx tsc --noEmit`. Clean exit. Clicking a result does nothing yet.

Step LC-16 — Inserting the token

Goal: Make clicking a result actually insert a `@[Name](id)` token at the right spot.

File: `MentionTextarea.tsx`

What we add:

TSX

```
import { useEffect, useRef, useState } from "react";
```

TSX

```
  const [results, setResults] = useState<{ id: number; name: string }[]>([]);
  const textareaRef = useRef<HTMLTextAreaElement>(null);

  function selectUser(user: { id: number; name: string }) {
    if (mentionStart === null || !textareaRef.current) return;
    const cursor = textareaRef.current.selectionStart ?? value.length;
    const token = `@[${user.name}](${user.id}) `;
    const newValue = value.slice(0, mentionStart) + token + value.slice(cursor);
    setValue(newValue);
    setQuery(null);
    setMentionStart(null);
  }
```

TSX

```
              <button
                type="button"
                onClick={() => selectUser(user)}
                className="w-full rounded px-2 py-1 text-left text-sm hover:bg-gray-100
dark:hover:bg-gray-700"
              >
```

Also add `ref={textareaRef}` to the `<textarea>` itself.

How it works. This is the real payoff of everything we set up in LC-13. We need to replace *exactly* the partial `@query` the user just typed — nothing before it, nothing after it.

TSX

```
value.slice(0, mentionStart) + token + value.slice(cursor)
```

Think of the comment text as three pieces:

TEXT

```
[ everything before the @ ] [ the @query being replaced ] [ everything after the cursor ]
        value.slice(0, mentionStart)                                value.slice(cursor)
```

- `value.slice(0, mentionStart)` keeps everything BEFORE the `@` — this is why `mentionStart` had to point at the `@` itself (from LC-13), not at the first letter of the query. If we had sliced up to the cursor instead, we'd accidentally keep the `@` and the partial query in front of our new token.
- `token` is the new text we're inserting: `` `@${user.name} ` ``. Notice this stores BOTH the name and the id, in one string — more on exactly why in the Challenge homework question.
- `value.slice(cursor)` keeps everything AFTER the original cursor position — any text the user had already typed further along in the comment.

Put together, the `@query` in the middle is discarded, and the token takes its place.

Why we need it: This is the step where the mention picker becomes actually usable — before this, clicking a result did nothing.

What to test: No automated test — verify by hand: type `@` plus a few letters of a teammate's name, click the result, and confirm the text now reads `@[Full Name](id) `.

Step LC-17 — Restoring focus and the cursor (important!)

Goal: After picking someone from the dropdown, make sure the textarea gets focus back, and the cursor lands immediately after the inserted token — even if there was more text after the original cursor.

File: `MentionTextarea.tsx`

The problem we're solving: Clicking a dropdown button moves the browser's focus to that *button* — not back to the textarea. On top of that, there's a subtler issue:

TEXT

```
setValue(newValue)
does NOT update the real DOM <textarea> immediately.

React only SCHEDULES the update — it doesn't happen synchronously, right there in the click
handler.

So if we tried to call textareaRef.current.setSelectionRange(...) right after setValue(...),
we'd be setting the cursor position on the OLD textarea value, a split second before React
actually swaps in the new one.
```

That's why we can't just add `.focus()` and `.setSelectionRange()` directly inside `selectUser`. We need to wait until AFTER React has actually re-rendered with the new value — and that's exactly what a `useEffect` that depends on `value` gives us.

What we add:

TSX

```tsx
  const textareaRef = useRef<HTMLTextAreaElement>(null);
  const pendingCaretRef = useRef<number | null>(null);

  function selectUser(user: { id: number; name: string }) {
    if (mentionStart === null || !textareaRef.current) return;
    const cursor = textareaRef.current.selectionStart ?? value.length;
    const token = `@[${user.name}](${user.id}) `;
    const newValue = value.slice(0, mentionStart) + token + value.slice(cursor);
    pendingCaretRef.current = mentionStart + token.length;
    setValue(newValue);
    setQuery(null);
    setMentionStart(null);
  }

  useEffect(() => {
    if (pendingCaretRef.current === null || !textareaRef.current) return;
    const caret = pendingCaretRef.current;
    pendingCaretRef.current = null;
    textareaRef.current.focus();
    textareaRef.current.setSelectionRange(caret, caret);
  }, [value]);
```

How it works, step by step.

`pendingCaretRef` is a `useRef`, not a `useState`. Why does that matter here? Because a `ref` doesn't trigger a re-render when it changes, and we don't need it to — we're just using it as a little "sticky note" that says "once you finish re-rendering, please put the cursor here." A `ref`'s value also survives between renders without disappearing, which is exactly what we need: we write to it inside `selectUser` (before the re-render) and read it inside the `useEffect` (after the re-render).

In `selectUser`, we now calculate where the cursor SHOULD end up:

TSX

```tsx
pendingCaretRef.current = mentionStart + token.length;
```

`mentionStart` is where the token starts; `token.length` is how long the inserted token is. Add them together, and you get the exact index right after the token — regardless of whether there's more text after it. (The text after the cursor, `value.slice(cursor)`, is already safely preserved right after the token by `selectUser` itself — this effect's only job is to place the cursor, never to touch that trailing text again.)

Then the new `useEffect`:

TSX

```tsx
useEffect(() => {
  if (pendingCaretRef.current === null || !textareaRef.current) return;
  const caret = pendingCaretRef.current;
```

```
    pendingCaretRef.current = null;
    textareaRef.current.focus();
    textareaRef.current.setSelectionRange(caret, caret);
  }, [value]);
```

This effect depends on `value`, which means it only runs AFTER React has committed a new value to the DOM — exactly the timing we needed. Once it runs:

- `textareaRef.current.focus()` — gives real browser focus back to the textarea (remember, the click moved focus to the dropdown button instead).
- `textareaRef.current.setSelectionRange(caret, caret)` — places the cursor at `caret`.

Why `(caret, caret)` and not two different numbers? `setSelectionRange(start, end)` normally selects a RANGE of text, from `start` to `end`. Passing the same number for both means "select nothing — just put the cursor right here." If we passed two different numbers, we'd end up highlighting a chunk of the comment instead of just placing the cursor.

Why `.focus()` has to come before `.setSelectionRange()`: An unfocused textarea still quietly accepts a selection range internally, but the user's next keystroke won't go anywhere near it until the textarea is actually focused. Calling `.focus()` first makes sure the user can immediately keep typing right where the cursor landed.

Why we need it: Without this step, after picking a mention, the user would have to manually click back into the textarea and find the right spot to keep typing — a frustrating experience for something that should feel instant.

What to test by hand:

1. Type some text, then type `@query`, then select a user. Confirm the textarea has focus (not the dropdown), and typing one more character immediately appears right after the inserted token.
2. Repeat with text ALREADY present after your original cursor position, and confirm that trailing text is still there, unchanged, right after the token.

Step LC-18 — Wire `MentionTextarea` into `CommentForm`

Goal: Replace the plain textarea in `CommentForm` with our new, smarter one.

File: `CommentForm.tsx`

What we add:

TSX

```
import { createCommentAction, type CommentFormState } from "@/lib/actions/comments";
import { MentionTextarea } from "@/components/ui/MentionTextarea";
```

TSX

```
export function CommentForm({
  issueId,
  workspaceId,
  basePath,
```

```
}: {
  issueId: number;
  workspaceId: number;
  basePath: string;
}) {
```

TSX

```
      <MentionTextarea name="body" workspaceId={workspaceId} required />
```

How it works: `MentionTextarea` already renders its own internal `<textarea name="body">`, so `createCommentAction`'s `formData.get("body")` read needs zero changes — only how that one field is *rendered* changed, not how the form submits.

A heads-up: `CommentForm` now REQUIRES a `workspaceId` prop that nothing passes in yet. This is a deliberate, temporary broken state — `tsc` will show an error right now, and the very next step fixes it immediately. Don't worry if you see a red squiggly line here; it's expected for about thirty seconds.

Step LC-19 — Pass `workspaceId` through; LIVE is complete

Goal: Fix the error from LC-18 by actually passing `workspaceId` into `CommentForm`.

File: `.../issues/[issueId]/page.tsx`

What we add:

TSX

```
      <CommentForm
        issueId={issue.id}
        workspaceId={project.workspaceId}
        basePath={`/workspaces/${id}/projects/${projectId}/issues/${issueId}`}
      />
```

How it works: `project.workspaceId` was already being fetched on this exact page (the members fetch needed it too) — so this step needs zero new data fetching, just passing an existing value one level deeper.

What to test:

BASH

```
curl -s --cookie "session=$TOKEN" http://localhost:3001/workspaces/1/projects/1/issues/1 \
  | grep -o "mention someone"
```

Expected result: `tsc` / `build` both pass cleanly, and the placeholder text mentioning `@mention`s shows up. LIVE is now complete.

How Comments and Mentions fit into Issue Detail

By the end of LIVE, the Issue Detail page has grown two new sections at the bottom: a list of comments (`CommentList`, showing mentions as nice pills) and a form to add a new one (`CommentForm`, with the full `@mention` autocomplete inside it). Nothing about the rest of the page — the issue title, status, priority,

assignee — changed at all.

The full LIVE data flow

TEXT

```
User types @na
↓
handleChange runs (LC-13)
↓
TRAILING_MENTION_RE checks the text right before the cursor
↓
query = "na", mentionStart = position of the @
↓
useEffect calls searchUsersAction(workspaceId, "na", true)  (LC-14)
↓
backend returns matching workspace members
↓
dropdown renders those results  (LC-15)
↓
user clicks "Nawar"
↓
selectUser() builds "@[Nawar](12) " and splices it into the text  (LC-16)
↓
React re-renders the textarea with the new value
↓
useEffect notices the new value and restores focus + cursor position  (LC-17)
↓
user finishes typing and submits the form
↓
createCommentAction sends the comment to the backend  (LC-02/LC-03)
↓
backend stores the comment, including the raw "@[Nawar](12)" token
↓
later, MentionText renders that token as a styled "@Nawar" pill  (LC-09/LC-10)
```

That's the whole LIVE session, start to finish. If any one piece of this chain feels unclear, go back to that step above and re-read the "How it works" section — each one was written to stand on its own.